Universidad de la Fuerzas Armadas - ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Análisis y Diseño de Software - NRC: 30723

Modelado de casos de uso de nivel 0 y apoyo al análisis orientado a objetos con PlantUML

Grupo 6:

Andrade Lucio Danna Valentina

Llumiquinga Ñacato Ariel Jose

Pilaguano Chisaguano David Alexander

Profesor: Ing. Jenny Ruiz

Fecha: 28 de Abril de 2026

GUÍA PARA DESARROLLAR EN PLANTUML DIAGRAMAS DE CASOS DE USO DE ANÁLISIS

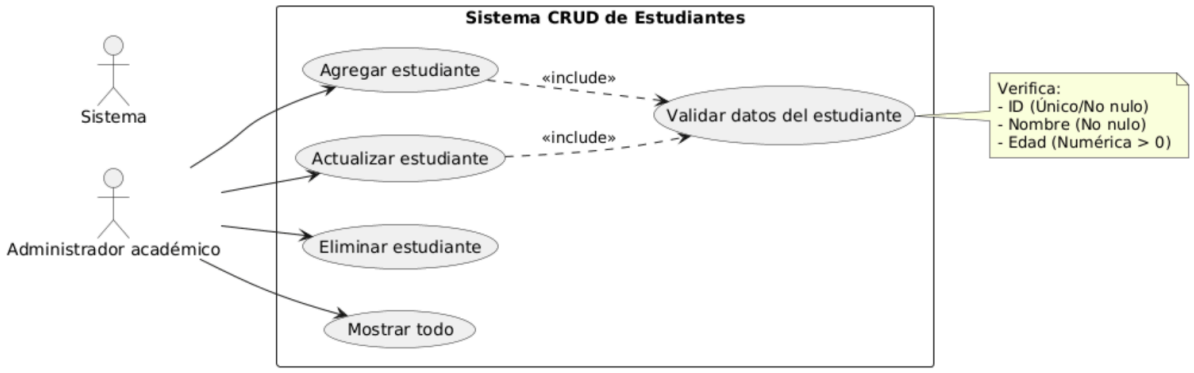
1. Requisitos Funcionales - Nivel 0

Código	Requisito Funcional	Prioridad
RF-01	El sistema debe permitir registrar un nuevo estudiante.	Alta
RF-02	El sistema debe permitir la actualización de datos de estudiantes existentes.	Alta
RF-03	El sistema debe permitir la eliminación de registros de estudiantes.	Media
RF-04	El sistema debe listar la totalidad de estudiantes registrados en una tabla.	Alta
RF-05	El sistema debe validar que los campos obligatorios no estén vacíos antes de procesar.	Alta

2. Diagrama de casos de uso

Código PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema CRUD de Estudiantes" {
  usecase "Agregar estudiante" as UC1
  usecase "Actualizar estudiante" as UC2
  usecase "Eliminar estudiante" as UC3
  usecase "Mostrar todo" as UC4
  usecase "Validar datos del estudiante" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>

note right of UC5
Verifica:
- ID (Único/No nulo)
- Nombre (No nulo)
- Edad (Numérica > 0)
end note
@enduml
```



3. Matriz de Trazabilidad RF-CU

RF	Caso de Uso	Actor	Prioridad	Criterio de Aceptación	Evidencia (UC)
RF-01	Agregar estudiante	Administrador	Alta	El registro se guarda sólo si el ID no existe previamente.	UC1
RF-02	Actualizar estudiante	Administrador	Alta	Se permiten cambios en Nombre/Edad tras validar el ID existente.	UC2
RF-03	Eliminar estudiante	Administrador	Media	El registro desaparece de la tabla tras confirmación.	UC3
RF-04	Mostrar todo	Administrador	Alta	Se visualizan todos los registros en formato de tabla (ID, Nombre, Edad).	UC4
RF-05	Validar datos	Sistema	Alta	El sistema lanza una alerta si faltan datos o la edad es inválida.	UC5

4. Conclusión técnica

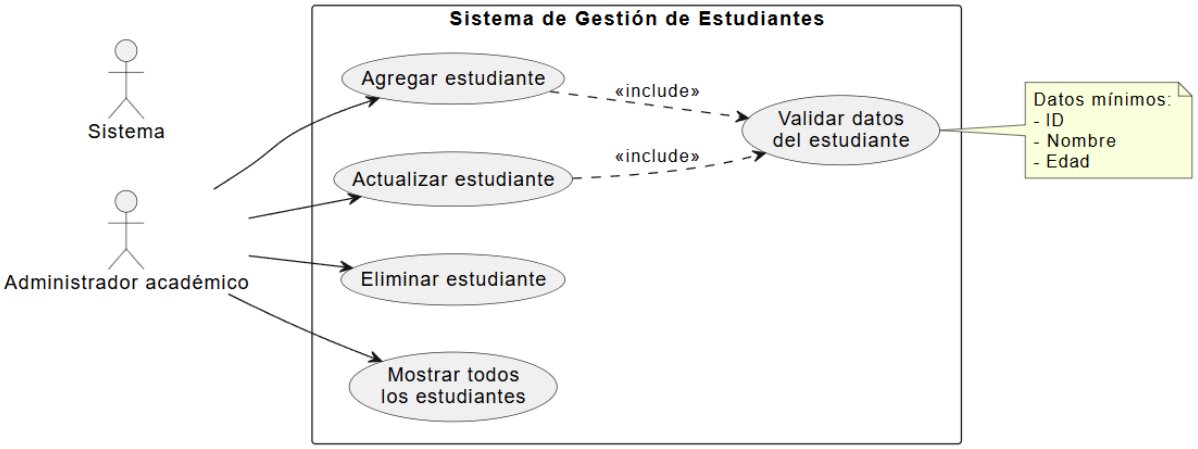
La integración de modelos permite verificar que el sistema no se desarrolle de forma fragmentada, sino como un conjunto donde todo está correctamente unificado para su funcionamiento óptimo. En el ejercicio del CRUD de Estudiantes, los casos de uso definen las necesidades del usuario, mientras que las clases de análisis organizan los elementos que deben existir y los diagramas de secuencia muestran la comunicación entre ellos para ejecutar cada tarea. Lograr esta trazabilidad es fundamental porque asegura que el diseño del sistema sea consistente, facilitando que el paso hacia la programación sea mucho más ordenado y sin errores de lógica.

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

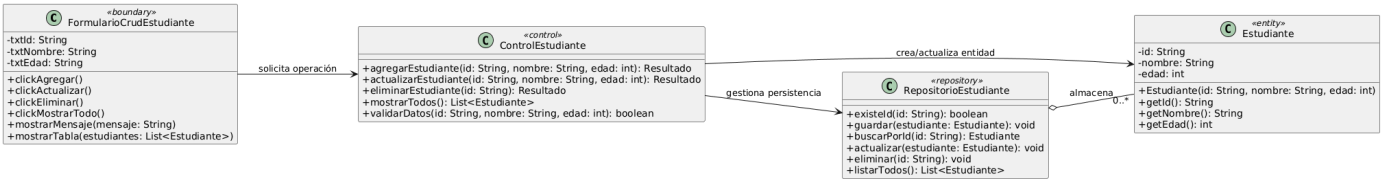
1. **Sección de casos de uso**
   Código PlantUML

```
@startuml
 left to right direction
 skinparam packageStyle rectangle
 skinparam shadowing false

 actor "Administrador académico" as Admin
 actor "Sistema" as Sistema

 rectangle "Sistema de Gestión de Estudiantes" {
   usecase "Agregar estudiante" as UC1
   usecase "Actualizar estudiante" as UC2
   usecase "Eliminar estudiante" as UC3
   usecase "Mostrar todos\nlos estudiantes" as UC4
   usecase "Validar datos\ndel estudiante" as UC5
 }

 Admin --> UC1
 Admin --> UC2
 Admin --> UC3
 Admin --> UC4

 UC1 ..> UC5 : <<include>>
 UC2 ..> UC5 : <<include>>

 note right of UC5
 Datos mínimos:
 - ID
 - Nombre
 - Edad
 end note
 @enduml
```



2. **Clases de Análisis**
   Código PlantUML

```
@startuml
 skinparam classAttributeIconSize 0
 skinparam shadowing false
 left to right direction

 class "FormularioCrudEstudiante" <<boundary>> {
   - txtId: String
   - txtNombre: String
   - txtEdad: String
   + clickAgregar()
   + clickActualizar()
   + clickEliminar()
   + clickMostrarTodo()
   + mostrarMensaje(mensaje: String)
   + mostrarTabla(estudiantes: List<Estudiante>)
 }

 class "ControlEstudiante" <<control>> {
   + agregarEstudiante(id: String, nombre: String, edad: int):
Resultado
   + actualizarEstudiante(id: String, nombre: String, edad: int):
Resultado
   + eliminarEstudiante(id: String): Resultado
   + mostrarTodos(): List<Estudiante>
   + validarDatos(id: String, nombre: String, edad: int): boolean
 }

 class "Estudiante" <<entity>> {
   - id: String
   - nombre: String
   - edad: int
   + Estudiante(id: String, nombre: String, edad: int)
   + getId(): String
   + getNombre(): String
   + getEdad(): int
 }

 class "RepositorioEstudiante" <<repository>> {
   + existeId(id: String): boolean
   + guardar(estudiante: Estudiante): void
   + buscarPorId(id: String): Estudiante
   + actualizar(estudiante: Estudiante): void
   + eliminar(id: String): void
   + listarTodos(): List<Estudiante>
 }

 FormularioCrudEstudiante --> ControlEstudiante : solicita operación
 ControlEstudiante --> Estudiante : crea/actualiza entidad
 ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
 RepositorioEstudiante o-- "0..*" Estudiante : almacena
 @enduml
```

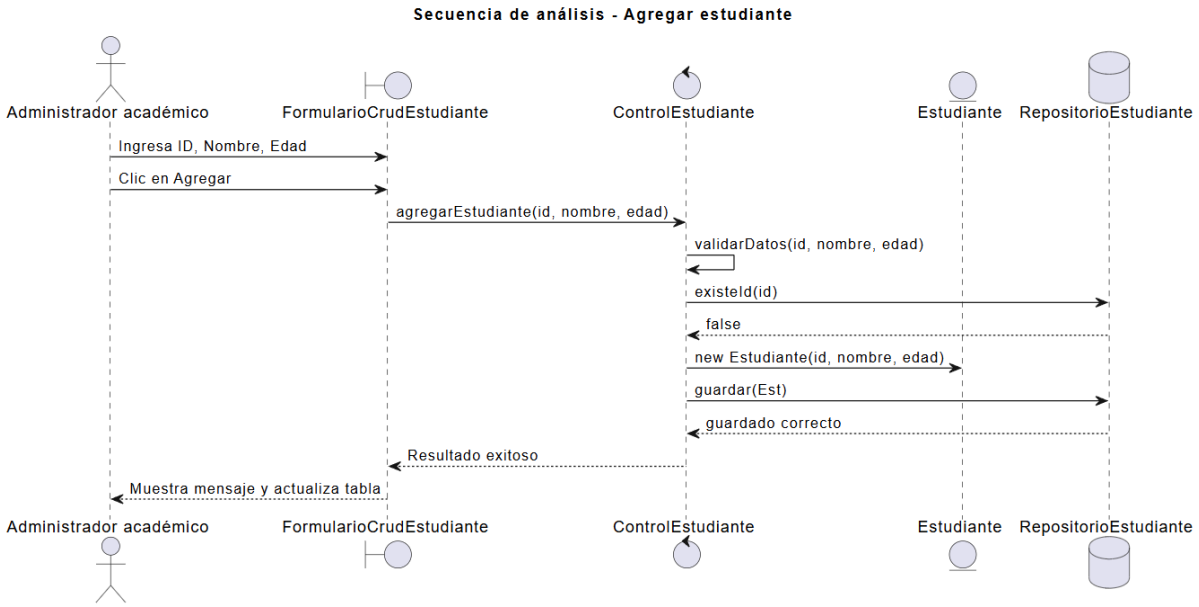


3. Secuencias de Análisis

1. Secuencia: Agregar estudiante

Código PlantUML

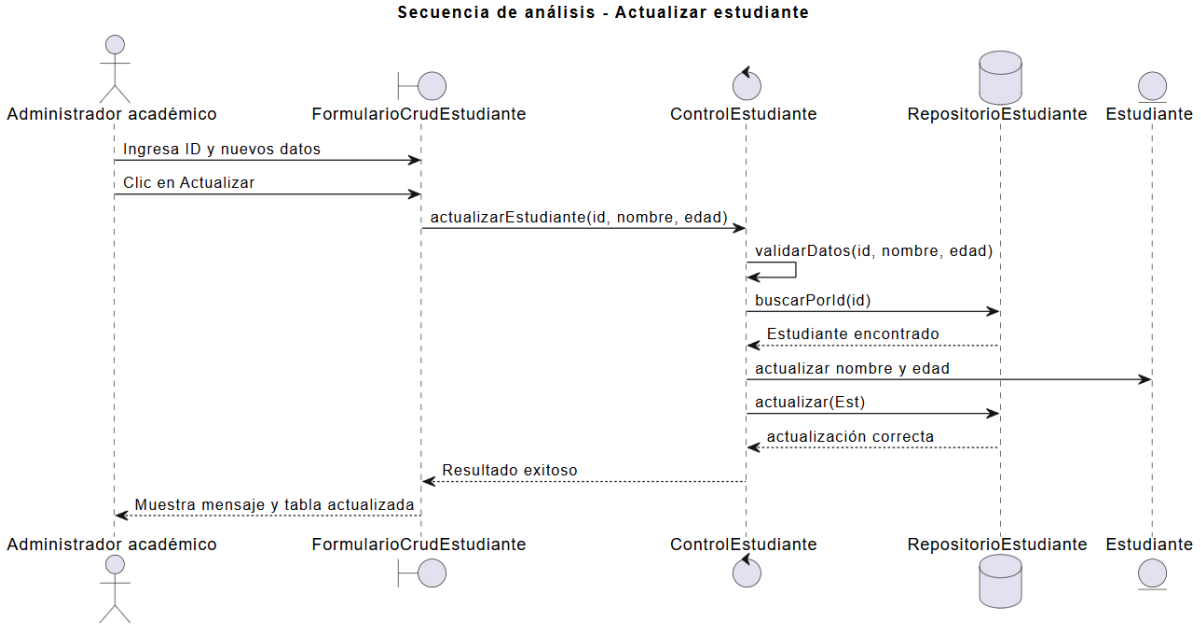
```
@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```



2. Secuencia: Actualizar estudiante

Código PlantUML

```
@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
```

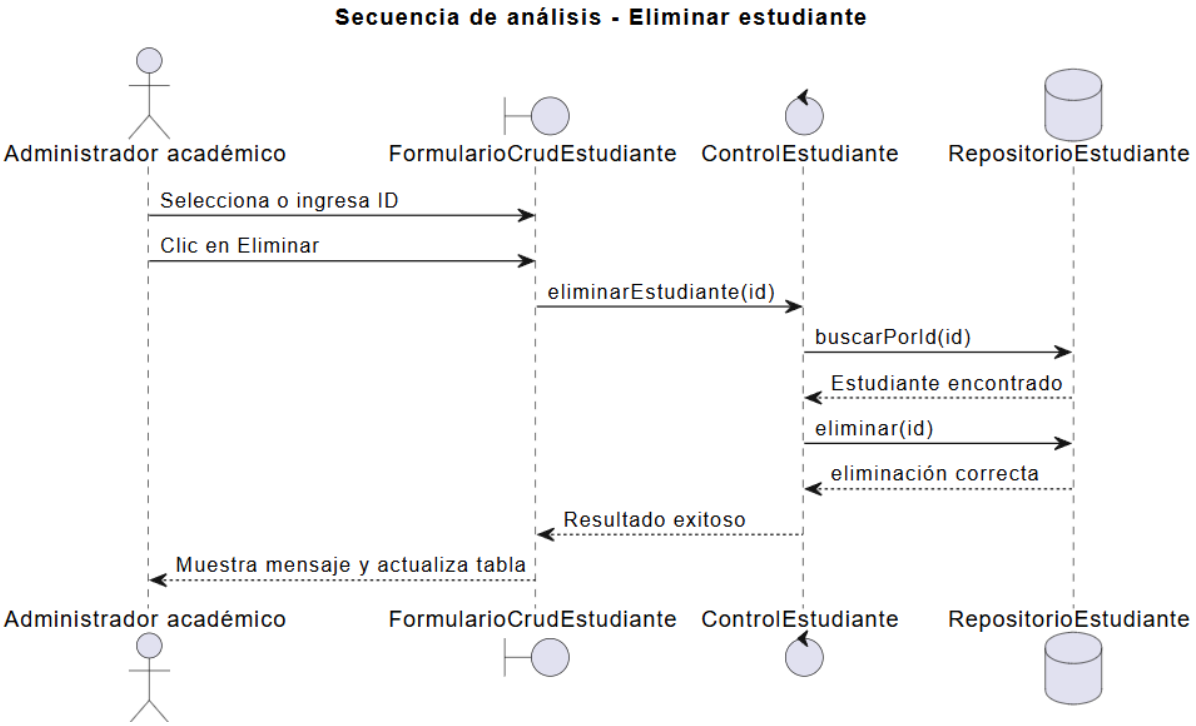


3. Secuencia: Eliminar estudiante

Código PlantUML

```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
```

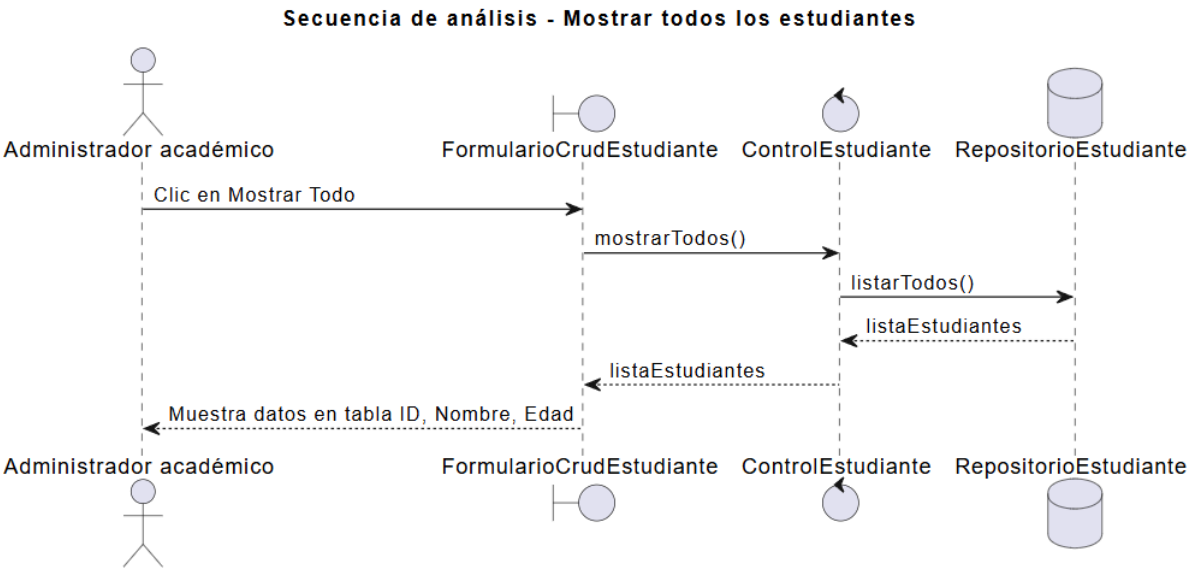
```
Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```



4. Secuencia: Mostrar todos los estudiantes

Código PlantUML

```
@startuml
title Secuencia de análisis - Mostrar todos los estudiantes
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
```



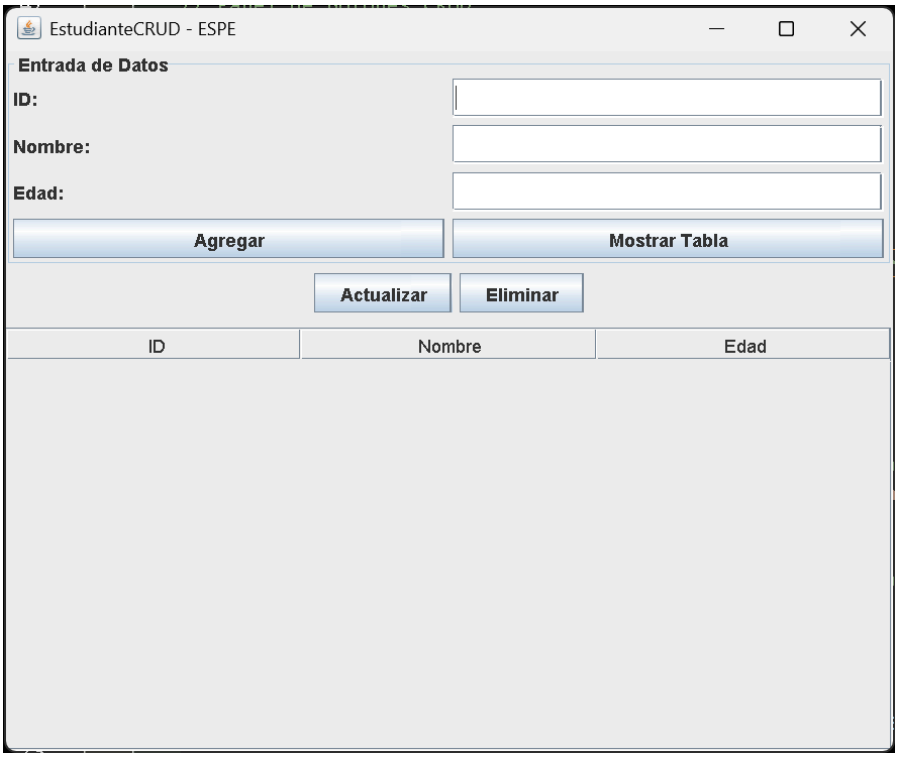
Trazabilidad

Requisito funcional (RF)	Caso de Uso (CU)	Clase Frontera (Boundary)	Clase control	Entidad/Repositorio	Diagrama de Secuencia
RF-01: Agregar un estudiante con ID, Nombre y Edad	**UC1:** Agregar estudiante.	FormularioCrudEstudiante	ControlEstudiante	Estudiante/RepositorioEstudiante	Secuencia Agregar estudiante
RF-02: Actualizar datos de un estudiante existente.	**UC2:** Actualizar estudiante	FormularioCrudEstudiante	ControlEstudiante	Estudiante/RepositorioEstudiante	Secuencia Actualizar estudiante.
RF-03: Eliminar un estudiante existente	**UC3:** Eliminar estudiante	FormularioCrudEstudiante	ControlEstudiante	RepositorioEstudiante	Secuencia Eliminar estudiante
RF-04: Mostrar todos los estudiantes registrados	**UC4:** Mostrar todos los estudiantes	FormularioCrudEstudiante	ControlEstudiante	RepositorioEstudiante	Secuencia Mostrar todos los estudiantes
RF-05: Validar datos obligatorios antes de cambios	**UC5:** Validar datos del estudiante	FormularioCrudEstudiante	ControlEstudiante	N/A	Flujo interno en secuencias Agregar estudiante y Actualizar estudiante

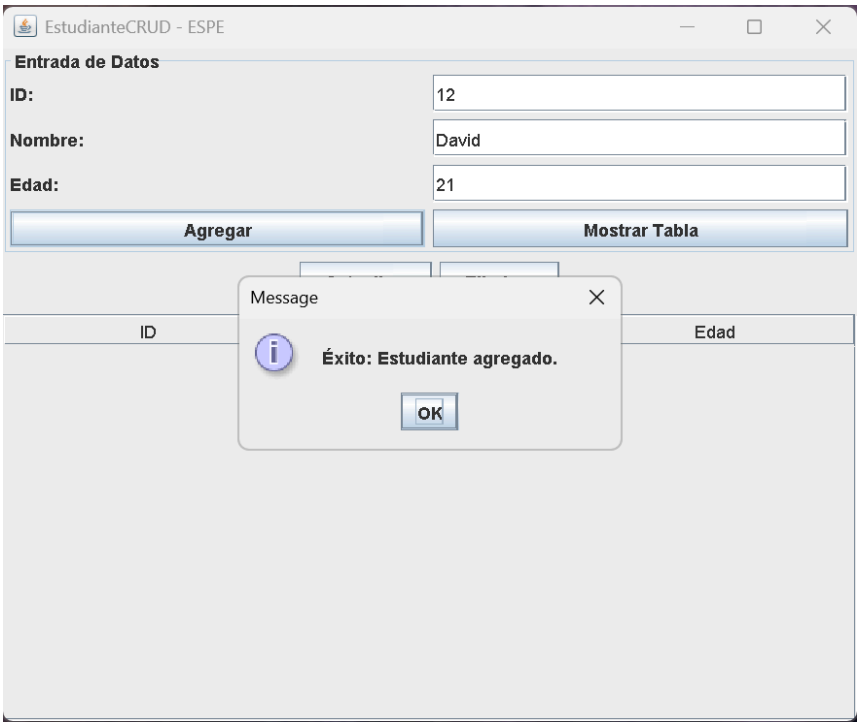
Conclusión técnica

La integración técnica de los casos de uso, las clases de análisis y los diagramas de secuencia permite validar que el sistema de gestión de estudiantes no se construya como un conjunto de piezas aisladas, sino como un modelo unificado y coherente. Mientras los casos de uso explican detalladamente qué necesita el actor del sistema , las clases de análisis definen qué objetos colaboran (frontera, control, entidad y repositorio) y las secuencias ilustran cómo interactúan estos componentes para cumplir con cada operación del CRUD. La trazabilidad técnica mejora significativamente la claridad del análisis y proporciona una base sólida y verificable para el diseño y la posterior implementación del sistema en Java.

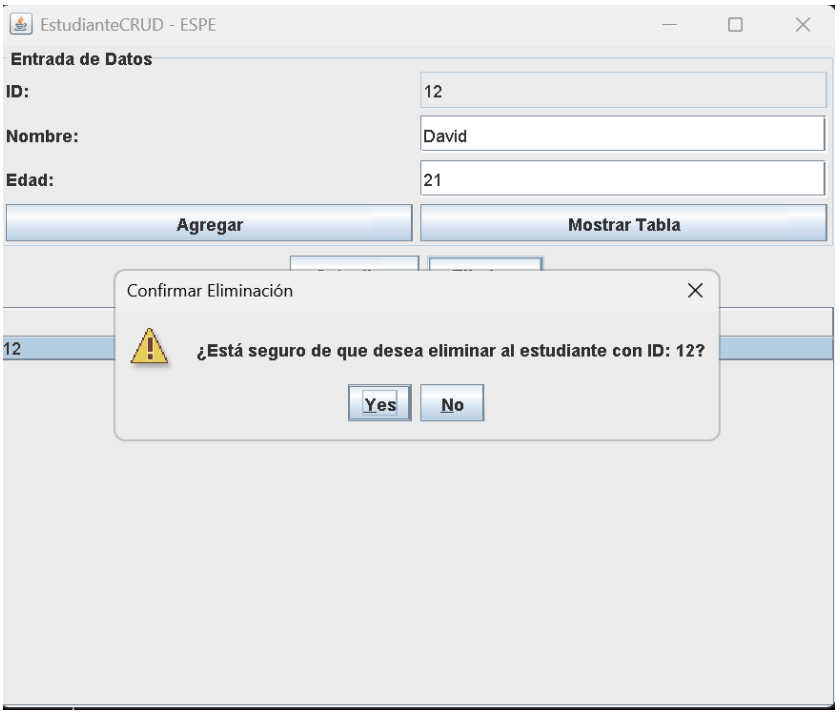
Anexos
Link del repositorio:
Código del ejercicio:
https://github.com/Dval05/AndradeDanna_30723_G6_ADSW/tree/main/Parcial%201/Talleres/Taller2/CodeEstudiante/EstudianteCRUD
Pantalla del CRUD
1.-



2.-


EstudianteCRUD - ESPE
Entrada de Datos
ID:
12
Nombre:
David
Edad:
21
Agregar
Mostrar Tabla
Message
Éxito: Estudiante agregado.
OK
ID
Edad

3.-


EstudianteCRUD - ESPE
Entrada de Datos
ID:
12
Nombre:
David
Edad:
21
Agregar
Mostrar Tabla
Confirmar Eliminación
¿Está seguro de que desea eliminar al estudiante con ID: 12?
Yes
No
12